

```
abstract class soyut { //soyut(abstract) sınıf (eksik/tamamlanmamış sınıf)
    void m1() { //concrete method
    }
}
```

```
    abstract void m2();
}
```

```
class bbb extends soyut { //somut sınıf
    @Override
    void m2 () {
        System.out.println("Merhaba");
    }
}
```

//somut bir sınıf soyut bir sınıftan miras alıyorsa o sınıftaki soyut sınıfları somut hale getirmek zorundadır

```
abstract class ccc extends soyut {
    int topla(int a, int b) {
        return a + b;
    }
}
```

```
abstract class ddd extends soyut {
}
```

```
class eee extends soyut {
    void m2() {
    }
}
```

```
abstract class x {
    x() { //soyut sınıfın constructoru da olabilir
        System.out.println("Merhaba");
    }
    int x, y;
    void m3(int c) {
        c = 5;
    }
}
```

```
class y extends x { //constructor chaining
    y() { //gizliiden super(); ♦airısı yapılır
        System.out.println("Dünya");
    }
    //abstract void m3(int c); //bunun ♦alınması i♦in y nin de abstract olması gerekir
}
```

```
abstract class z extends x {  
    abstract void m4(int c); //reddi miras vakası(mirası bırakmayı reddetme)  
}
```

```
class w{  
    void m3(int c){  
  
    }  
}
```

```
public class okul11 {  
    public static void main(String[] args) {  
  
        new y();  
  
    }  
}
```

```
//abstract class and interfaces  
//bir soyut method ya bir soyut sınıfta bulunabilir ya da bir interface'de  
//bulunabilir  
//soyut metodlar static tanımlanamaz  
//soyut sınıftan nesne üretilemez  
//soyut sınıflar soyut metod bulundurmamak zorunda değil  
//bir soyut sınıfın atası somut sınıf olabilir  
//sınıf diyagramında soyut sınıf ve soyut metodlar italik yazılır  
//kare işareti protected modifier olduğunu söyler  
//
```